

Systèmes d'exploitation I

Module: M3 - INF-S3 & IAP-S3

Prof. R. HANNANE

r.hannane@uca.ma

Faculté des Sciences Senglalia, Marrakech
Université Cadi Ayyad



Systèmes de fichiers de Linux

Quelques informations utiles

Notes à propos des **fichiers** :

- Un **fichier** est un **ensemble de données enregistrées de façon à être lues et traitées par ordinateur**
- Les fichiers **peuvent être regroupés dans des répertoires**
- Un **répertoire** peut contenir soit des fichiers, soit d'autres répertoires dans une **structure arborescente**
- Les **appels système** permettent de créer les fichiers et les répertoires, ainsi que de les supprimer, de les ouvrir, de les lire et de les modifier

Quelques informations utiles

Types de fichiers

- Le système **Linux ne connaît que 3 types de fichiers**:
 - ❶ **Les fichiers ordinaires(-)**: contiennent des données,(ex: fichiers de texte, binaires, images...). Ils servent à mémoriser les programmes et les données des utilisateurs et du système
 - ❷ **Les fichiers répertoires(d)**: sont les **nœuds de la structure arborescente** des fichiers. Ils correspondent à des sous-bibliothèques ou dossiers qui contiennent d'autres fichiers
 - ❸ **Les fichiers spéciaux**: ils sont **liés à un périphérique et correspondent à :**
 - ▶ **des programmes(b,c)** (pilotes)qui gèrent les échanges avec les périphériques (disques, terminaux, imprimantes, lecteurs de bandes, etc)
 - ▶ **des tubes et sockets(p,s)**, utilisés pour les liaisons interprocessus (la communication entre processus)
 - ▶ **des liens symboliques(l)** (fichiers "pointant" sur un autre fichier)

Quelques informations utiles

Nomenclature des fichiers

- Pour donner un nom à un fichier, il faut suivre quelques règles simples
- Ces règles sont valables pour tous les types de fichiers:
 - ▶ Sur les anciens systèmes Unix **un nom de fichier ne pouvait pas dépasser 14 caractères**. Sur les systèmes actuels, dont Linux, **on peut aller de 1 jusqu'à 255 caractères**
 - ▶ Linux fait la **distinction entre les noms de fichiers en minuscules et en majuscules** (ex: Toto, TOTO, ToTo et toto sont des noms de fichiers différents, avec un contenu différent)
 - ▶ **La plupart des caractères** (les chiffres, les lettres, les majuscules, les minuscules, certains signes, les caractères accentués) **sont acceptés, y compris l'espace**
 - ▶ **Quelques caractères sont à éviter** car ils ont une signification particulière au sein du shell : `& ; ( ) ~ / | ' ?`

Systèmes de fichiers de Linux

- Un **système de fichiers** est une structure logique qui permet au système d'exploitation d'accéder aux fichiers qui se trouvent dans un périphérique de données (Disque dur, clé USB, etc).
- Il est **créé au moment de l'opération de formatage**
⇒ il correspond au **type de formatage** qui va être appliqué à une partition,
(*Il définit la manière dont les fichiers sont nommés, stockés et récupérés sur un périphérique de stockage*)
- Les systèmes d'exploitation utilisent des systèmes de fichiers différents tel que *FAT, ExFAT, NTFS, HFS, HFS+, APFS, Ext4,...*
⇒ **L'organisation et le fonctionnement de ces systèmes de fichiers est différent** ou certains peuvent être plus performants dans des dossiers avec des milliers de fichiers, d'autres dans le recouvrement dans le cas de corruption...
- **Ext** est le système de fichiers par défaut sur les systèmes Linux
⇒ **Ext4** gère parfaitement de nombreux petits fichiers

Systèmes de fichiers de Linux

Système de fichiers hiérarchisés

- **Le système de fichiers de Linux est à géométrie variables**, il est compatible avec la plupart des systèmes de fichiers existant (et entre autre les systèmes FAT, FAT 32 ou NTFS de Windows de Microsoft), **il peut gérer la plupart des support physiques, et il peut s'agrandir à tout moment** (l'administrateur du système peut créer un nouveau point de montage n'importe où dans l'arborescence)
- **L'organisation du système de fichiers est une organisation logique basée sur une organisation physique constituée d'un ou plusieurs disques**
 - ▶ Chacun est divisé en une ou plusieurs partitions physiques et l'espace disque physique disponible est ainsi découpé en **disques logiques**
 - ▶ Ces disques sont découpés en série de groupe de blocs, ou chaque groupe de blocs est composé de la même manière
 - ▶ Les groupe de blocs du disque stockent les **inodes** (les métadonnées des fichiers et dossiers comme le type de fichiers, permissions, etc) ainsi que les **données du fichier**

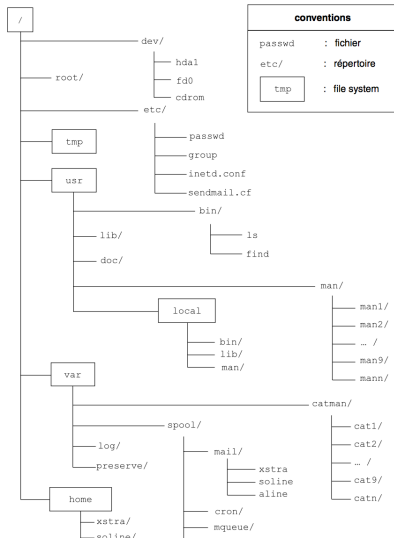
Systèmes de fichiers de Linux

Système de fichiers hiérarchisés

- Initialiser un disque logique consiste à créer sur ce disque un système de fichiers appelé **file system**
- **L'arborescence de fichiers** est construite au démarrage du système à partir de plusieurs **file system**
- En général, **plusieurs systèmes de fichiers** sont "**montés**" sur le **système "racine"**
⇒ Celui qui contient le répertoire " / "
- Un file system particulier, le **root file system** contient la racine de l'arborescence, le noyau, les fichiers systèmes, et des répertoires vides (*/tmp/, /usr/, ... sur lesquels seront greffées des arborescences de fichiers se trouvant dans d'autres file systems*)

Systèmes de fichiers de Linux

Organisation hiérarchisée



Systèmes de fichiers de Linux

Système de fichiers hiérarchisés

Répertoire	Description
/	Répertoire racine, point d'entrée au système de fichiers
/boot	Contient le noyau Linux et l'amorceur
/bin	Contient les exécutables de base:ls,cp,mv...
/dev	Contient des fichiers spéciaux représentant les périphérique de la machine
/etc	Contient les fichiers de configuration du système
/home	Contient les fichiers des utilisateurs(répertoire par utilisateur)
/lib	Contient les librairies et les modules du noyau (/lib/modules)
/media	Point de montage des média (CD,USB...)
/root	Répertoire personnel de l'administrateur
/sbin	Contient les exécutables relatifs à l'administration du système
/tmp	Contient les fichiers temporaires
/usr	Contient les exécutables des programmes(/usr/bin et /usr/sbin) la documentation (usr/doc)
/var	Contient les fichiers relatifs à la maintenance du système

Systèmes de fichiers de Linux

Structure d'un file system

- La structure d'un file system est globalement la même dans toutes les versions d'Unix/Linux: Chaque file system est composé d'un grand nombre de blocs contigus
- Sa structure de base est comme suite :

Bloc	Contenu
0	contient le boot et l'identification du disque
1	appelé aussi <i>superbloc</i> contient des informations sur le file system (<i>date de dernière mise à jour du file system, le nombre de fichiers qu'il peut contenir, sa taille, un pointeur sur les blocs libres</i>)
2 à k	contiennent la table des inodes
à partir du bloc k+1	sont stockées les blocs de données

Systèmes de fichiers de Linux

Les inodes

- Sous Unix/Linux, un **fichiers est une suite d'octets non structurée**
- ⇒ **Toutes les informations sur le fichier qui permettent de garder la trace de tous les fichiers et répertoires d'un système de fichiers basé sur Linux ou UNIX, sont regroupées dans une structure de données appelée **inode****
- **Un fichier n'est pas repéré par son nom mais par le numero (**index**) de l'inode qui le caractérise**
 - ⇒ Dans **ext4** de Linux, le nom de fichier est stocké dans un bloc de répertoire Ce dernier contient le numéro d'inode, le nom et la longueur du nom du fichier.
- **Cet index variant de 1 à n permet de retrouver l'inode dans la **table des inodes****
 - ⇒ Aucun fichier n'utilise le numéro 1
 - ⇒ Le numéro 2 représente la racine du file system
- **Le nombre maximum de fichiers par file system est égal au nombre d'inodes alloués à l'initialisation**

Systèmes de fichiers de Linux

Les inodes

- Les informations contenues dans un inode sont:
 - 1 **Bloc de disque** où la donnée est stockée
 - 2 **Type de fichier**: (*fichier ordinaire, répertoire,...*)
 - 3 **Bits de permissions**: `suid,sgid,t,rwx rwx rwx`
 - 4 **Nombre de liens**
 - 5 **Numéro de propriétaire (UID)**
 - 6 **Numéro de groupe (GID)**
 - 7 **Taille du fichier en octets**
 - 8 **Des pointeurs** vers les blocs de données correspondants
 - 9 **Date et heure du dernier accès**
 - 10 **Date et heure de la dernière modification**
 - 11 **Date et heure de la dernière modification de l'inode**

Systèmes de fichiers de Linux

Les inodes: création et fonctionnement

- Le système de fichiers **lors du formatage d'une partition de disque en ext4 crée chaque inode** qui possède un numéro unique au sein du système de fichiers
- **Le nombre total d'inodes d'un système de fichiers est défini manuellement lors de sa création**
⇒ Il indique le nombre maximal de fichiers qu'un système de fichiers peut supporter.
- **Lors de la création ou de la copie d'un fichier, Linux attribue un numéro d'inode différent au nouveau fichier**
- **Lors du déplacement d'un fichier, le numéro d'inode ne change que si le fichier est déplacé vers un système de fichiers différent**

Systèmes de fichiers de Linux

Les chemins

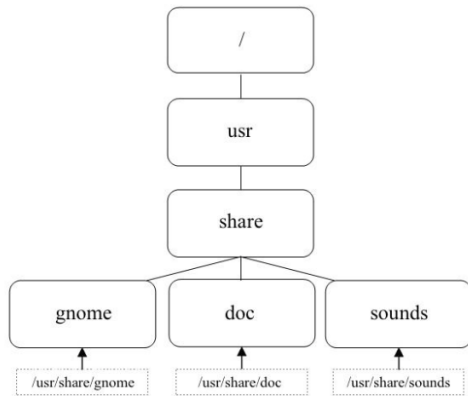
- Un **chemin** permet de définir un emplacement au sein du système de fichiers
- Tous les fichiers peuvent être **atteints par un chemin** dans la hiérarchie à partir du dossier racine $\Rightarrow$ c'est la liste des répertoires et sous-répertoires empruntés pour accéder à un fichier dans un endroit donné: **de l'arborescence jusqu'à la position souhaitée** (répertoire, fichier)
- Dans chaque répertoire, on trouve au moins deux fichiers, nommés **.** (point) et **..** (point point)
 - ▶ Le premier, qui a pour symbole **.** permet de référencer le répertoire lui-même (répertoire courant)
 - ▶ Le second, qui a pour symbole **..** permet d'accéder au **répertoire parent** (répertoire situé juste au dessus)
- A chaque instant, toute tâche possède un répertoire courant, ou répertoire de travail

Systèmes de fichiers de Linux

Les chemins

- **Exemple:** On prend la figure suivante:
On remarque qu'il y a un répertoire nommé "sounds" qui se trouve dans le répertoire "share" qui se trouve lui-même dans le répertoire "usr" qui se trouve lui-même dans "racine"

⇒ Pour accéder au répertoire "sounds" on spécifie son chemin comme suit:
`/usr/share/sounds`
où **le premier "/"** représentent **le répertoire racine**, **les autres "/"** sont utilisés pour **séparés les noms des répertoires**



Systèmes de fichiers de Linux

Les chemins

- Un **nom de fichier** est ainsi généralement **complété par son chemin d'accès**

⇒ C'est ce qui fait que le fichier *toto* du répertoire *rep1* est différent du fichier *toto* du répertoire *rep2*

- Pour désigner un fichier quelconque, on peut utiliser soit un :
 - a) **chemin absolu**
 - b) **chemin relatif**

Systèmes de fichiers de Linux

Les chemins: a) chemin absolu

- Un **chemin absolu** permet d'accéder à un fichier quelconque dans l'arborescence du système de fichiers
- Il est composé d'une suite de noms de répertoires à traverser en partant de la racine, séparés par le caractère / (et non \ comme sous DOS) et se termine par le nom du fichier que l'on veut atteindre
- La **longueur du chemin d'accès absolu** d'un fichier est **limitée à 1024 caractères**

Systèmes de fichiers de Linux

Les chemins: a) chemin absolu

A retenir:

- **Un chemin absolu ou complet :**

- ▶ **Démarre de la racine**, donc commence par un /,
- ▶ **Décrit tous les répertoires à traverser** pour accéder à l'endroit voulu
- ▶ **Ne contient pas de . ni de ..**

⇒ **Example:**

Le chemin " /usr/bin/compress" désigne le fichier *compress*, qui se trouve dans le répertoire *bin*, lui-même dans le répertoire *usr* de la racine

Le premier caractère / indique qu'il s'agit d'un chemin absolu

Systèmes de fichiers de Linux

Les chemins: b) chemin relatif

- Un nom de **chemin** peut aussi être **relatif** à sa **position courante** dans le répertoire
- Le **système** (ou le **shell**) mémorise la **position actuelle** d'un utilisateur dans le **système de fichier**, le répertoire actif
- On peut accéder à un autre répertoire de l'arborescence depuis l'emplacement actuel sans taper le chemin complet
⇒ **Uniquement en précisant le chemin le plus court relativement à votre position actuelle au sein de l'arborescence**
- Il faut souvent **utiliser deux entrées** particulières de **répertoires** :
 - ▶ **Le point .** représente le répertoire courant, actif. Il est généralement implicite
 - ▶ **Les doubles points ..** représentent le répertoire de niveau inférieur

Systèmes de fichiers de Linux

Les chemins: b) chemin relatif

A retenir:

- Un chemin relatif :
 - ▶ **Décrit un chemin relatif à une position donnée** dans l'arborescence, généralement (mais pas toujours) **depuis la position courante**
 - ▶ **Décrit en principe le plus court chemin** pour aller d'un point à un autre
 - ▶ **Peut contenir le point . ou les doubles points ..**

Connexion/Déconnexion d'un utilisateur en Linux

Connexion d'un utilisateur

- Le **système Linux** étant un **système multi-utilisateur et multitâche** : $\Rightarrow$ plusieurs personnes sont connectées simultanément et peuvent travailler sans interférer les unes avec les autres
- Linux met en œuvre des mécanismes **d'identification des utilisateurs**, tout en permettant le **partage contrôlé** nécessaire au travail en groupe
- Tout utilisateur est **identifié par un nom** (*login name*)
- Cet utilisateur ne peut utiliser le système que si son nom a préalablement été défini par **l'administrateur du système** (ou super-utilisateur), dont le nom est généralement **root**

Connexion d'un utilisateur

- Lorsque on installe Linux, et pour ouvrir une session, on a l'opportunité d'indiquer si on veut utiliser un **écran graphique** (**connexion en mode graphique**) plutôt qu'une **console** (**connexion en mode texte**)
- Pour qu'un utilisateur puisse travailler avec le système Linux, **il doit établir une connexion qui consiste à s'identifier auprès du système** ⇒ **ouvrir une session**:
 - ▶ **Login**: saisir le nom d'utilisateur
 - ▶ **Password**: saisir son mot de passe

Attention:

- **Les caractères saisis ne sont pas affichés à l'écran** (on dit qu'il n'y a **pas d'écho des caractères sur le terminal**)
- Ce mécanisme permet de garder la confidentialité du mot de passe

Connexion d'un utilisateur

- En cas d'erreur lors de la saisie du nom ou du mot de passe, le système donne à l'utilisateur la possibilité de recommencer
- Si la connexion est réussie, il apparaît alors à l'écran un **certain nombre d'informations** (informations générales, date, arrivée de messages, date de dernière connexion)
- Puis le système lance un programme qui est généralement un **interpréteur de commandes**

Déconnexion d'un utilisateur

- Pour sortir de la session, on peut :
 - Utiliser la commande `exit` si on est sur un environnement de lignes de commande
⇒ *La déconnexion est effective lorsqu'apparaît l'invite login :*
 - Sélectionner avec le bouton gauche de la souris le menu démarrer puis choisir la commande `log out`
⇒ *La déconnexion est effective lorsqu'apparaîtra l'invite graphique login*

Connexion/Déconnexion d'un utilisateur

Conseils généraux

Remarques:

- Si vous avez réalisé vous-même l'installation, une fois cette dernière terminée, **connectez-vous au système en tant qu'administrateur** (répondre root au login :)
- Il faut immédiatement définir un mot de passe pour cet utilisateur **root** (Ce mot de passe doit contenir plus de six caractères)
- Ce mot de passe vous permettra de vous connecter en tant qu'utilisateur **root** et ainsi d'avoir tous les droits sur le système

Connexion/Déconnexion d'un utilisateur

Conseils généraux

Attention:

- Le compte **root** est réservé pour le superviseur du système
- **Il ne faut jamais utiliser root comme un compte personnel**
- Il doit être utilisé pour réaliser des modifications dans **votre système** $\Rightarrow$ *C'est pourquoi la première action d'un administrateur est de créer un compte personnel*
- Un utilisateur peut à tout moment changer son mot de passe mais, il faut fournir l'ancien

L'interpréteur de commandes

- Après l'ouverture de la session, l'utilisateur se trouve sous le contrôle d'un **interpréteur de commandes**
- C'est un logiciel système faisant partie des composants de base d'un SE
- Sa fonction est d'interpréter les commandes qu'un utilisateur tape au clavier dans l'interface en ligne de commande
⇒ **Shell** est l'interpréteur qui traduit les commandes entrées par l'utilisateur en actions que le noyau doit exécuter
- Il existe plusieurs types de Shell mais le plus utilisé par les distributions Linux est **bash**
- Cet interpréteur de commandes est prêt à lire, analyser et éventuellement exécuter les commandes qui lui sont soumises

L'interpréteur de commandes

- **L'interpréteur indique par une chaîne de caractères** (marqueur au début de ligne), appelée **invite** (ou **prompt**), qu'il est prêt à recevoir une commande
- **Le marqueur du début de ligne est variable selon les machines**

Exemple: \$ ou *nom_utilisateur@nom_machine* >

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ █
```

- **Le caractère de terminaison peut avoir d'autres significations:**
 - un \$ indique que **l'utilisateur n'a pas de pouvoirs particuliers**, comme le >
 - un # indique que **l'utilisateur est l'administrateur root qui a tous les pouvoirs**

L'interpréteur de commandes

Répertoire de connexion

- Après le login, **l'interpréteur de commande** a pour répertoire courant le **répertoire de connexion** de l'utilisateur
- Sous le répertoire racine **“/”** on trouve un répertoire appelé **“home”**
- Sous le répertoire **“home”** il y aura un **répertoire pour chaque utilisateur dans le système**
- Ce répertoire qui porte le nom de l'utilisateur est appelé **répertoire personnel de l'utilisateur** ou (home directory)
Exemple : Un utilisateur appelé “abdessamad” aura comme répertoire personnel **“/home/abdessamad”**

L'interpréteur de commandes

Répertoire de connexion

- **L'utilisateur a un contrôle total sur son répertoire personnel**
⇒ L'utilisateur peut y placer ses fichiers personnels, et créer autant de sous-répertoires qu'il le désire
- **Vous pouvez représenter votre répertoire personnel par le symbole "*~*" dans l'invite de commande au lieu d'utiliser le chemin complet**
Si votre répertoire personnel est */home/admin* vous pouvez l'écrire aussi *~*
- Pour représenter le répertoire personnel d'un autre utilisateur on peut taper *~user* (*user est le nom de l'utilisateur*)
Par exemple : *~abdessamad* représente le répertoire personnel de abdessamad (*/home/abdessamad*)